

Абелевы группы

Необходимо реализовать класс конечно порождённой абелевой группы `AbelianGroup`. Абелева группа G строится как фактор свободной группы $\mathbb{Z}^n$ по соотношениям, то есть по порождающим ядра факторизации. Лабораторная работа доступна на двух языках: Python и C++. Можно писать на любом из них, разбалловка и тесты одинаковы для обоих языков.

Задания не являются независимыми: следующие методы могут (и будут) опираться на предыдущие. В частности, без `smith_normal_form` и `AbelianGroup.invariant_factors` реализовывать остальные методы бессмысленно.

Сдавайте файл с вашим решением в [classroom](#) лабораторных работ, дедлайн — **23:59 20 мая**. По всем вопросам пишите в телеграм Максиму Малкову [@Serious_Meow](#). Максим есть в чатах всех групп по алгебре, поэтому первым делом стоит задавать вопросы в них. Так вы сможете найти ответы на вопросы, заданные ранее, а ответ на ваш вопрос может помочь другим.

Ниже приводятся пояснения для каждого из языков.

Python

В лабораторной используется python-библиотека `SymPy`, родственная системе компьютерной алгебры `SageMath`. Она же будет использоваться в следующей лабораторной для работы с многочленами. Рекомендуется использовать python версии ≥ 3.11 . От `SymPy` нам нужны целочисленные матрицы, см. документацию <https://docs.sympy.org/latest/tutorials/intro-tutorial/matrices.html>.

Для выполнения работы и запуска тестов стоит создать виртуальное окружение и установить в него зависимости. В некоторых IDE это можно сделать через родной интерфейс или соответствующее расширение. Если ваша среда этого не умеет, ниже приводятся команды для настройки через командную строку из папки с работой (Linux, MacOS):

```
python3 -m venv .venv          # создание окружения
source .venv/bin/activate      # активация
python3 -m pip install -r requirements.txt # установка зависимостей
python3 -m pytest              # запуск тестов
```

Тесты находятся в файле `test_public.py`, можете менять их и добавлять новые по своему усмотрению, но при оценивании будут запускаться все тесты, находившиеся в файле изначально.

На случай проблем с `pytest` к работе приложен ноутбук `testing.ipynb`, в котором можно запустить тесты вручную. Также для выполнения тестов можно запустить `test_public.py` как python-скрипт:

```
python3 test_public.py
```

Но в обоих случаях не будет оформленного вывода статистики и ошибок, а тестирование будет идти до первого провалившегося теста.

Лабораторная работа решается в файле `solution.py`, он же сдаётся на проверку. В файле решения вам нужно внести свои данные в `PersonalInfo` строго как в ведомости.

Некоторые пояснения по `SymPy`:

- векторы — только столбцы, `Matrix([a,b]) == Matrix([[a],[b]])`, обе имеют размеры (shape) 2×1 .
- столбцы матрицы `M`: `[M.col(i) for i in range(M.cols)]`
- Для операций над строками и столбцами (для функции `smith_normal_form`) можно использовать методы `elementary_col_op` и `elementary_row_op`, см. <https://docs.sympy.org/latest/modules/matrices/matrices.html#operations-on-entries>.

- У матриц есть метод `diagonal()`, который, к сожалению, падает на матрицах, у которых одна из размерностей 0 (т.е. 0 столбцов, например). Проверяйте этот граничный случай, пожалуйста.
- Нетривиальные методы класса матриц SymPy (вроде разложения матрицы или метода Гаусса) использовать нельзя. Можно:
 - взятие определителя (`.det()`)
 - обратной матрицы (через `.inv()` или через возведение в (-1) -ю степень: `M**(-1)`)
 - вещи вроде `sympy.lcm(a,b)` (НОК пары чисел) и `sympy.igcd(a,b)` (НОД пары чисел)
 - конструкторы стандартных матриц вроде `eye`, `zeros`, `diag`
- Пользоваться стандартными библиотеками, такими как `functools` и `itertools`, не возбраняется.

C++

В лабораторной работе используется библиотека Eigen. Зафиксирована версия C++ 20. От Eigen нам нужны целочисленные матрицы, см. документацию https://eigen.tuxfamily.org/dox/classEigen_1_1Matrix.html.

В качестве элементов матрицы используется тип `int64_t`, для размеров используется тип `ssize_t`. Размеров этих типов достаточно для проведения расчётов без переполнения. Также в заголовочном файле `abelian_group.hpp` объявлены типы матрицы, вектора и списка инвариантных множителей.

Для сборки лабораторной работы достаточно инструментов, использовавшихся на курсе по C++: CMake и Git. Для инициализации создайте директорию в папке с материалами (например, `build`) и в ней выполните:

```
cmake .. # без отладчика
# или
cmake -DCMAKE_BUILD_TYPE=Debug .. # с отладчиком
```

Для компиляции и запуска тестов в этой же папке выполните цель `run-tests`:

```
make run-tests
```

для ускорения первой сборки можно добавить флаг `-j<num>`, где `<num>` - число потоков (достаточно использовать число ядер вашего процессора):

```
make run-tests -j8
```

Тесты находятся в файле `test_public.cpp`, для их создания как и на курсе по C++ используется библиотека Catch2, работа с ней должна быть вам знакома.

Лабораторная работа решается в файле `solution.cpp`, он же сдаётся на проверку. В файле решения вам нужно внести свои данные в `personal_info` строго как в ведомости.

Некоторые пояснения по Eigen:

- Матрицы инициализируются построчно
- Для создания вектора нужно передать в конструктор его длину, а потом заполнить значениями. Например, $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$ - это `Vector v{2}; v << 1, 2;`
- Количество столбцов в матрице `M.cols()`, количество строк `M.rows()`
- Доступ к элементу в строке `r` и столбце `c`: `M(r, c)`
- Можно получить доступ к строке `r` через `M.row(r)` и к столбцу `c` через `M.col(c)`. Операции со строками и столбцами векторизированные, то есть запись `M.row(a) -= M.row(b) * c` вычитает из строки `a` строку `b`, умноженную на `c`. Для перестановки можно использовать `M.row(a).swap(M.row(b))`
- Нетривиальные методы для работы с матрицами использовать нельзя, можно:

- Взятие определителя (`M.determinant()`)
- Взятие обратной матрицы. Так как `Eigen` не поддерживает это для целочисленных матриц, в решении определена функция `inverse`, которая считает целочисленную обратную матрицу
- Создание стандартных матриц, вроде `Matrix::Identity`, `Matrix::Zero`
- Разрешается пользоваться всей стандартной библиотекой, например, `std::lcm` (НОК) и `std::gcd` (НОД)

Реализуйте следующие методы:

- (5 баллов) функция `smith_normal_form(A)`:

Для ненулевой целочисленной матрицы A размера $n \times k$ возвращает тройку S, D, T , где

- D — диагональная матрица $n \times k$ неотрицательных целых чисел, в которой каждый ненулевой диагональный элемент делится на предыдущий; в частности, нулевые элементы диагонали, если они есть, расположены ниже ненулевых.
- S и T — целочисленные матрицы с целочисленными обратными размеров $n \times n$ и $k \times k$ соответственно,
- $D = S \cdot A \cdot T$.

- (1 балл) `AbelianGroup.invariant_factors()`:

Возвращает список инвариантных множителей $u_1, \dots, u_m$ (являются натуральными числами), дополненный нулями до длины `self.n`. Нули соответствуют свободным множителям, см. следствие 1 лекции 5.

- (0.5 балла) `AbelianGroup.is_trivial()`:

Возвращает `True`, если группа тривиальна (т.е. состоит из нейтрального элемента), и `False` иначе.

- (0.5 балла) `AbelianGroup.is_cyclic()`:

Возвращает `True`, если группа циклическая (тривиальная считается циклической), и `False` иначе.

- (0.5 балла) `AbelianGroup.is_free()`:

Возвращает `True`, если группа свободная, и `False` иначе.

- (0.5 балла) `AbelianGroup.rank()`:

Возвращает ранг группы, если она свободная, и `None` иначе.

- (0.5 балла) `AbelianGroup.is_finite()`:

Возвращает `True`, если группа конечная, и `False` иначе.

- (0.5 балла) `AbelianGroup.size()`:

Возвращает размер группы, если она конечна, и 0 иначе.

- (2 балла) `AbelianGroup.ord(v)`:

Возвращает порядок смежного класса вектора v , если он конечен, и 0 иначе.

- (2 балла) `AbelianGroup.tor()`:

Возвращает подгруппу кручения как абстрактную группу (то есть произвольную абелеву группу, изоморфную подгруппе кручения).

Оценивание

Для каждого метода прохождение одноимённого теста даёт указанное количество баллов. Полученная сумма нормируется до 10 (делением на 1.3). К итогу прибавляются бонусные 0.5 балла в случае прохождения дополнительного теста `test_zero_dim` для группы $\mathbb{Z}^0$.